

Explanation for the Sentivity Analysis code

Yunru Zheng

November 8, 2025

1 Finding the gradient

We are doing sensitivity analysis on the std of our importance function $g(x) = N(0, \sigma)$. We will find the gradient of the variance with respect to σ , and use statistical approximation to find the optimal sigma value by gradient descent.

Since the variance is

$$v(\sigma) = \text{var}_\sigma(L(x, \sigma)X^{2n}) = E_\sigma[(L(x, \sigma)X^{2n})^2] - E_\sigma[L(x, \sigma)X^{2n}]^2,$$

and the second term is a constant independent of the choice of σ , calculating $\partial v / \partial \sigma$ is basically taking partial derivative with respect to $E_\sigma[(L(x, \sigma)X^{2n})^2]$.

I basically manually calculated $(L(x, \sigma)X^{2n})^2$ and did a change of variables $x = \sigma\epsilon$ since then I can take the partial derivative directly in the integral. I set

$$h(\epsilon, \sigma) = (L(\sigma\epsilon, \sigma)(\sigma\epsilon)^{2n})^2,$$

then we get

$$E_\sigma[(L(x, \sigma)X^{2n})^2] = E_f[h(\epsilon, \sigma)],$$

since it's easier to take the partial derivative now:

$$\frac{\partial v}{\partial \sigma} = \frac{\partial}{\partial \sigma} E_\sigma[(L(x, \sigma)X^{2n})^2] = \frac{\partial}{\partial \sigma} E_f[h(\epsilon, \sigma)] = E_f[\nabla_\sigma h(\epsilon, \sigma)].$$

Thus, maximizing the gradient of h maximizes the variance after importance sampling. I calculate by hand that

$$h(\epsilon, \sigma) = \sigma^{4n+2} \epsilon^{4n} e^{\epsilon^2(1-\sigma^2)}, \quad \nabla_\sigma h(\epsilon, \sigma) = h(\epsilon, \sigma) \left(\frac{4n+2}{\sigma} - 2\epsilon^2 \sigma \right)$$

Note: I realized that in my code I used k instead of n since that's what I used in the last coding exercise. Sorry about that!

2 Stochastic Approximation

According to Asmussen and Glynn and the last section, we are minimizing the function $E[h(\epsilon, \sigma)]$. It is smooth and for a minimizer σ^* , $\nabla_\sigma E[h(\epsilon, \sigma^*)] = 0$. Unfortunately it's very hard to calculate and make this value 0, we instead look at $Y_{n+1} = \nabla_\sigma h(\epsilon, \sigma_{n+1})$, since $E[Y_{n+1}] \approx \nabla_\sigma E[h(\epsilon, \sigma_n)]$. thus, my formula for finding σ_{n+1} is:

$$\sigma_{k+1} = \sigma_k - s_k \nabla_\sigma h(\epsilon, \sigma_n)$$

During my run, I set a lower bound of 0.75 for σ because I need to make sure that

$$[E_\sigma[(L(x, \sigma)X^{2n})^2]] = \int \sigma^2 e^{x^2(\frac{1}{\sigma^2}-1)} x^{4n} \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}} dx = \int \sigma^2 e^{x^2(\frac{1}{2\sigma^2}-1)} x^{4n} dx$$

is finite, which means that the exponent has to be negative to balance out X^{2n} , which means that $\frac{1}{2\sigma^2} - 1 < 0$, and thus $\sigma > \sqrt{\frac{1}{2}}$.

The results isn't very ideal. I only did it for $n = 1$, so X^2 . I did learning rate at 0.0001, 0.0005, and 0.001 iterating 50000 times in total. Although all of them briefly stopped at the optimal sigma value $\sqrt{3}$, they all deviated from that value later on during the run. It's really hard to determine the optimal sigma value just by looking at the data without knowing the optimal sigma value beforehand. Also based on my assumption, $h(\epsilon, \sigma)$ is supposed to be minimized when $\sigma = \sqrt{3}$ is optimal, instead it's minimized near the boundary $\sigma = 0.75$. This means that I cannot determine the optimal σ by looking for the minimum $h(\epsilon, \sigma)$.

I think the rapid approach moving on with sigma just after one calculation increases the uncertainty and the fluctuation would be less if we use a batched value for $\nabla_\sigma h(\epsilon, \sigma_n)$ in each step.

As a result, I didn't do further runs the code on $n \geq 1$. I instead calculated the variance of $\nabla_\sigma h(\epsilon, \sigma^*)$ at the optimal sigma value. It turned out that since the function depends on ϵ^{4n} , the larger n is, the bigger the variance is. The noise is very big at large n for the gradient estimator to converge and stabilize at an optimal sigma value.